

Fundamentals of Artificial Intelligence and Knowledge Representation – Module 4

Prof. Federico Chesani – 7th of July, 2020

Available time: 25 min. + 10 min. for submitting the essay

Notice: when submitting your essay, please name the file in the following way:

FamilyNameFirstName.date.pdf/.docx

For example, if you are John Smith, the file will be named:

SmithJohn.20200707.pdf

Exercise

The candidate is invited to present the notion of the cut “!” operator in the Prolog language, shortly explaining how it works. To this end, the candidate is invited to show a short example of a predicate containing the cut, and to illustrate its effects.

After that, the candidate is invited to write a meta-predicate

how_many(Pred, Num_of_clauses)

that, given in input a predicate **Pred**, returns in **Num_of_clauses** the number of of clauses defining the predicate **Pred** in the current program.

For example, given the program:

```
a(X) :- b(X), c(X).  
a(X) :- Y is X+1, b(Y), c(Y).  
a(X) :- Y is X-1, b(Y), c(Y).
```

and the query

```
?- how_many(a(X), Result),
```

the following outcomes are expected:

```
?- how_many(a(X), Result).  
Result = 3.
```

This is because there are three different clauses that defines the predicate **a(X)**.

Fundamentals of Artificial Intelligence and Knowledge Representation – Module 4

Prof. Federico Chesani – 7th of July, 2020

Available time: 25 min. + 10 min. for submitting the essay

Solution

The candidate is invited to present the notion of the cut “!” operator in the Prolog language, shortly explaining how it works. To this end, the candidate is invited to show a short example of a predicate containing the cut, and to illustrate its effects.

It is expected that the candidate reports a short explanation of the cut, with an example of a program (two clauses would do). See the slides for the details.

After that, the candidate is invited to write a meta-predicate

how_many(Pred, Num_of_clauses)

that, given in input a predicate Pred, returns in Num_of_clauses the number of definitions (of clauses) defining the specified predicate in the current program.

```
how_many(Pred, Num_of_clauses) :-  
    findall(Clause, clause(Pred, Clause), Clauses)  
    , length(Clauses, Num_of_clauses) .
```